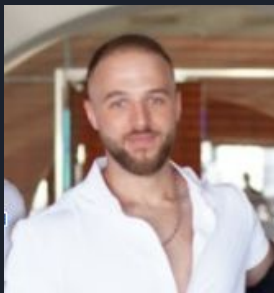




CI/CD and Jenkins

React Application with Express Server

Ido Carmi & Yuval Arviv



Ido Carmi

Telephone - Network Operation Center
25 Years old, live in Ramat Gan



Yuval Arviv

Telephone - Network Operation Center
23 Years old, live in Mazkeret batya

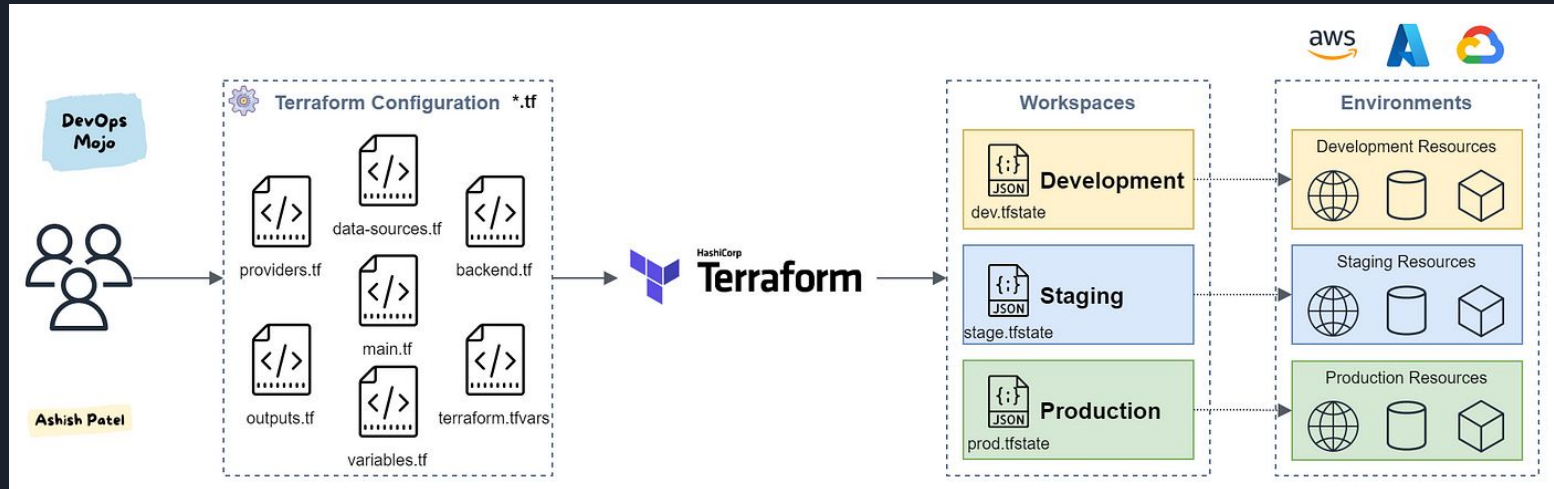
What is Jenkins

- **Automation Server:** Jenkins is an automation server that streamlines software development processes.
- **Extensible Plugins:** It's highly customizable and extends functionality through plugins.
- **Pipeline as Code:** Jenkins supports defining workflows as code, enhancing automation.
- **User-Friendly Interface:** Offers a web-based interface for easy job configuration.



What is Terraform

- **Infrastructure as Code:** Terraform is a tool used for provisioning and managing cloud infrastructure.
- **Multi-Cloud Support:** Terraform supports multiple cloud providers (e.g., AWS, Azure, Google Cloud) and other infrastructure platforms.
- **Plan and Apply:** Terraform provides a "plan" feature to preview changes before applying them, enhancing safety and predictability in infrastructure changes.





Project Overview

Goal: Create a seamless CI/CD pipeline for a React application with an Express server.

Setup: The project was initiated with Create React App, and an Express server was integrated in the server directory. The proxy key in package.json handles server proxying.

Objective: Achieve automated building, testing, and deployment processes to ensure efficient development and delivery.



Running the Application

- Clone the project
- Install dependencies
- Application runs on port 3000

```
yuval@yuval:~/Desktop/project/red-project$ ls
Jenkinsfile  README.md  frontend  package-lock.json  server  terraform  test
yuval@yuval:~/Desktop/project/red-project$
```

```
# Make port 3000 available to the world outside this container
EXPOSE 3000
```



NPM Install / Start

- Run 'npm install'
- Run 'npm start'

```
yuval@yuval:~/Desktop/red-project/server$ npm install  
  
up to date, audited 177 packages in 2s  
  
14 packages are looking for funding  
  run `npm fund` for details
```

```
yuval@yuval:~/Desktop/project/red-project/server$ npm start  
  
> react-express-starter@0.1.0 start  
> node index.js  
  
Express server is running on localhost:3000
```

Running Tests

- Change directory to test folder
- Install requirements with 'pip3 install -r requirements.txt'
- Run tests using 'pytest test/test.py'

```
yuval@yuval:~/Desktop/red-project$ pytest test/test.py
===== test session starts =====
platform linux -- Python 3.10.12, pytest-7.2.1, pluggy-1.0.0
rootdir: /home/yuval/Desktop/red-project
collected 2 items

test/test.py .. [100%]

===== 2 passed in 0.00s =====
```


Building Docker Images

- Build two Docker files (server and frontend)

frontend	frontend	49d7db6cb063
----------	----------	--------------

```
# Use an official Node runtime as the parent image
FROM node:14

# Set the working directory in the container to /app
WORKDIR /app

# Copy package.json and package-lock.json to the workdir
COPY package*.json ./

# Install any needed packages specified in package.json
RUN npm install
```

server	server	53df9c82936e
--------	--------	--------------

```
# Bundle app source inside the docker image
COPY . .

# Make port 3000 available to the world outside this container
EXPOSE 3001

# Run server.js when the container launches
CMD ["npm", "start"]
```

Setting up Jenkins CI Pipeline

- Configure Jenkins to serve the application
- Implement CI pipeline
- Run tests using pytest

```
Jenkinsfile M x
Jenkinsfile
1 pipeline {
2   // Define environment variables
3   environment {
4     // Image names for the server and frontend
5     SERVER_IMAGE = 'red_project:server'
6     FRONTEND_IMAGE = 'red_project_frontend'
7     // Docker Hub login credentials
8     DOCKER_USER = credentials('docker-hub-user')
9     DOCKER_PASSWORD = credentials('docker-hub-password')
10    // AWS credentials for Terraform
11    AWS_CREDENTIALS = credentials('aws-credentials')
12  }
13
14  // run on any available Jenkins agent
15  agent any
16  stages {
17    stage('Build') {
18      steps {
19        // Build steps for the server
20        dir('server') {
21          sh 'npm install'
22        }
23
24        // Build steps for the frontend
25        dir('frontend') {
26          sh 'npm install'
27        }
28      }
29    }
30  }
31}
```

Deploying with Terraform

- Use Jenkins to build Terraform CD
- Deploy two EC2 instances on AWS

```
yuval@yuval:~/Desktop/red-project/terraform$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

Instances (2) Info										
Find instance by attribute or tag (case-sensitive)										
Refresh Connect Instance state ▼ Actions ▼ Launch instances ▼										
< 1 > ⚙										
☐	Name ▼	Instance ID	Instance state ▼	Instance type ▼	Status check	Alarm status	Availability Zone ▼	Public IPv4 DNS ▼	Public	
☐	Frontend	i-0eaab35cf69c2ae36	✔ Running 🔍	t3.micro	✔ 2/2 checks passed	No alarms +	eu-north-1c	ec2-16-170-209-222.eu...	16.170	
☐	Server	i-021f72e0286dd96d8	✔ Running 🔍	t3.micro	✔ 2/2 checks passed	No alarms +	eu-north-1c	-	-	

Deploying Docker Images

- Deploy the first Docker image to the first application
- Deploy the second Docker image to the second application

TAG				docker pull yuvalarviv/red_proj_ 	
frontend					
Last pushed 2 hours ago by yuvalarviv					
DIGEST	OS/ARCH	LAST PULL	COMPRESSED SIZE		
0d33d9b10a57	linux/arm64/v8	---	444.17 MB		

TAG				docker pull yuvalarviv/red_proj_ 	
server					
Last pushed 2 hours ago by yuvalarviv					
DIGEST	OS/ARCH	LAST PULL	COMPRESSED SIZE		
a992901dce31	linux/arm64/v8	---	332.81 MB		

Conclusion

- The CI/CD pipeline automates the application deployment
- Jenkins, Docker, and Terraform streamline the process
- Efficiently manage and deploy your React-Express application.



Edit src/App.js and save to reload.

Enter your name:

Hello ido!

[Learn React](#)

Thank you
DevSecOps 12

